

Project Documentation: Student Grading System

A Python-Based Console Application for Academic Performance Tracking and Records Management

Language: **Python 3.x** Architecture: **Modular In-Memory System** Document Type: **Technical Summary Report**

1. Executive Summary

The **Student Grading System** is an interactive Python application designed to streamline academic record-keeping. The software automates score tracking across three primary academic subjects (Mathematics, Science, English), computes percentages, categorizes performance based on standard academic letter grades, and aggregates statistics for whole-class performance analysis.

Core Objective: Provide educational instructors and administrators with an intuitive, error-tolerant CLI platform to digitize student evaluation and extract instant academic analytics.

2. Key System Features & Capabilities

Student Registration

Captures student name, unique roll number, and subject marks. Includes robust `try-except` exception handling to reject non-numeric inputs.

Automatic Evaluation

Dynamically computes total marks (out of 300), overall average percentages, and maps scores to grade tiers ranging from **A+** to **Fail**.

Targeted Record Search

Allows instant lookup of individual student profiles via unique roll numbers, presenting comprehensive individual reports.

Class Analytics Engine

Calculates macro-level insights including total enrolled count, class average percentage, and automatically flags top performers.

3. Grading Criteria Matrix

Grade distribution is calculated algorithmically using the total calculated percentage across all recorded subjects:

Percentage Range	Grade Assigned	Academic Status	Description
90% and above	A+	Outstanding	Demonstrates superior subject mastery and accuracy.
80% – 89.9%	A	Excellent	Exceeds standard learning objectives consistently.
70% – 79.9%	B	Good	Meets standard academic requirements solidly.
60% – 69.9%	C	Satisfactory	Acceptable comprehension with room for growth.
50% – 59.9%	D	Pass	Meets minimum qualifying threshold.
Below 50%	Fail	Unsatisfactory	Requires remedial academic intervention.

4. Architecture & Implementation Highlights

The codebase is constructed around modular functions utilizing dynamic in-memory data structures (lists of dictionaries) for data storage and manipulation.

```
# Grade Mapping Logic Implementation
def get_grade(per):
    if per >= 90: return "A+"
    elif per >= 80: return "A"
    elif per >= 70: return "B"
    elif per >= 60: return "C"
    elif per >= 50: return "D"
    else: return "Fail"

# Record Creation and Data Ingestion
def add_student_data():
    s_name = input("Student Name: ")
    roll = input("Roll Number: ")
    try:
        maths_mark = float(input("Maths marks: "))
        sci_mark = float(input("Science marks: "))
        eng_mark = float(input("English marks: "))
    except Exception:
        print("Error: Enter numbers only for marks!\n")
    total_marks = maths_mark + sci_mark + eng_mark
    percent = total_marks / 3
    final_grade = get_grade(percent)
    students_list.append({
        "name": s_name,
        "roll_no": roll,
        "maths": maths_mark,
        "science": sci_mark,
        "english": eng_mark,
        "total": total_marks,
        "percentage": percent,
        "grade": final_grade
    })
```

5. System Workflow & Data Flow

- Command-Line Interface (CLI) Loop:** Operates inside an infinite while True loop, directing user operations via standard numerical selection (Choices 1–5).
- Data Handling Safety:** Implements type checking during entry to prevent runtime crashes caused by unformatted text input.
- Storage Lifetime:** Records remain stored in memory throughout runtime for standard record display, queries, and analytics generation.

6. Analytical Capabilities & Summary Statistics

The analytics module (show_stats()) aggregates overall class metrics dynamically upon request:

Metric	Calculation Logic	Operational Value
Total Enrollment	len(students_list)	Monitors real-time dataset capacity and cohort size.
Class Average Score	sum(percentages) / total_students	Evaluates whole-class academic health and curriculum alignment.
Top Performer	max(percentage) iteration check	Identifies highest achiever for merit recognition.

7. Future Enhancements & Scalability

- Persistent Database Integration:** Incorporate SQLite or JSON file storage to retain records across application restarts.
- CSV Export Capabilities:** Enable instructors to export evaluation summaries for institutional reporting.
- Dynamic Subject Expansion:** Allow custom subject configurations beyond the default three-course setup.